PDF Tool

Full Professional Re-Audit

Post-migration audit — AWS EC2 deployment
Repository: github.com/mukeshroyhub/pdf_tool
Live: https://pdftool4u.duckdns.org

Date: 16 July 2026
Prepared for: Mukesh Kumar

OVERALL PRODUCTION READINESS	89 / 100

Up from ~87% at the last re-audit. The app is live on AWS with working HTTPS, auth, storage and email. Remaining gaps are operational (single-box resilience, monitoring) and hygiene (rotate exposed secrets), not functional.

1. Executive Summary

PDF Tool is a full-stack PDF web application (Next.js 15 / React 19 front end, Express + TypeScript API, Prisma ORM) that has been successfully migrated from Render to a self-managed AWS EC2 free-tier instance. The migration is complete and verified live: the health endpoint responds over HTTPS at the new domain, and the container logs show real uploads, downloads and annotations succeeding — confirming the managed database (Neon) and object storage (Supabase) are correctly wired.

The codebase is in good shape. Security fundamentals are strong: strict Helmet CSP, locked-down CORS, tiered rate limiting, magic-byte upload validation, hashed rotating refresh tokens in httpOnly cookies, and fail-fast environment validation. Five new features shipped since the last audit (page numbers, ZIP download, password protect/unlock, e-signatures) plus polish (dark mode, retention, auto-open viewer).

The principal risks are no longer in the code — they are operational. The app now runs on a single 1 GB instance with no monitoring, no alerting and no redundancy, and several production secrets were exposed in plain text during setup and must be rotated. These are addressed in the prioritized action plan (Section 8).

Scorecard vs. previous audit

Area	Score	Δ	Note
Architecture & structure	**9.0**	=	Clean monorepo; stateless VM design
Security	**8.5**	▲	Strong controls; secret rotation outstanding
Deployment / DevOps	**8.0**	▲▲	Now on EC2 + Caddy HTTPS; single point of failure
Code quality	**8.5**	=	Typed, layered, readable; runs via tsx in prod
Features & UX	**9.0**	▲	5 new tools + dark mode, retention, auto-open
Testing / CI	**7.0**	=	7 test suites + green CI; no E2E, lockfile dropped
Performance	**7.5**	=	Fine at low load; 1 GB RAM caps heavy PDF ops
Documentation	**9.0**	▲	Deploy guides, session log, this audit, new skill

Δ compares against the previous re-audit. ▲ improved, ▲▲ strongly improved, = unchanged.

2. Architecture & Stack

The project is an npm-workspaces monorepo with three packages: a Next.js web app (~7,250 LOC), an Express API (~3,650 LOC), and a shared package of Zod schemas and types that keeps the client and server contracts in lockstep. Prisma is the ORM, using driver adapters (libSQL for local dev, PostgreSQL for production).

- **Front end:** Next.js 15, React 19, TanStack Query, Radix UI, Tailwind, framer-motion, next-themes. App-router with grouped auth routes.
- **API:** Express 4, TypeScript, Prisma 6, JWT auth, Helmet, CORS, express-rate-limit, Multer uploads, pdf-lib / pdfjs-dist / @napi-rs/canvas / sharp for PDF work, qpdf for encryption, archiver for ZIP.
- **Managed services (off-box):** Neon PostgreSQL, Supabase S3-compatible storage, Brevo transactional email (HTTPS API), Google OAuth.

- **Infra:** AWS EC2 t3.micro (Ubuntu, 1 GB RAM, 30 GB disk, 3 GB swap), Docker Compose, Caddy reverse proxy with automatic Let's Encrypt HTTPS, DuckDNS domain.

Strength: the VM holds no state — database and files live on managed services — so the instance can be rebuilt or replaced at any time without data loss. This is the right design for a small deployment and makes disaster recovery simple.

3. What Changed Since the Last Audit

- **Platform migration:** moved off Render (which suffered cold-start sleeping and 502s on the free tier) to a self-managed EC2 box with always-on Caddy HTTPS.
- **New features:** page numbering, multi-file ZIP download, password protect / unlock (qpdf), and e-signatures (draw / type / upload).
- **UX polish:** dark mode, branded 404, quick-tools dashboard grid, auto-open viewer after upload, and a 60-minute data-retention policy with a dashboard banner.
- **Data safety:** removed `--accept-data-loss` from the Prisma push so a destructive schema change aborts instead of silently dropping data.
- **Email reliability:** switched from SMTP (blocked on many hosts) to Brevo's HTTPS API.
- **Footprint reduction:** OCR and Office-conversion removed to fit the 1 GB box.

4. Security Review

Strengths (verified in code)

- **Helmet CSP** with `default-src 'none'` and `frame-ancestors 'none'` — strong clickjacking / injection defense; `x-powered-by` disabled.
- **CORS** restricted to the configured web origin with credentials — not a wildcard.
- **Rate limiting** tiered: 300 req / 5 min general, 10 attempts / 15 min on credential endpoints.
- **Upload validation** by magic bytes (sniff.ts) — the client MIME type is only trusted after the on-disk signature agrees; unknown types rejected outright.
- **Auth:** bcrypt password hashing, short-lived JWT access tokens, rotating hashed refresh tokens in an httpOnly, Secure (prod), SameSite=Lax cookie scoped to `/api/auth`.
- **Config validation:** Zod schema fails fast on startup; JWT secrets must be ≥32 chars; S3 credentials required when storage driver is s3.
- **Secrets hygiene in repo:** no `.env` is tracked in git; only `.env.example` templates.

Issues & recommendations

#	Severity	Finding	Recommendation
S1	**CRITICAL**	Live secrets exposed in plain text during setup (DB password, Supabase keys, Brevo key, Google secret, DuckDNS token; GitHub PATs earlier).	Rotate every one of them now that the app is stable; revoke the GitHub tokens. Update .env on the box and redeploy.

S2	**MEDIUM**	Containers run as root (no USER directive in Dockerfiles).	Add a non-root user and drop privileges — limits blast radius if the app is compromised.
S3	**MEDIUM**	100 MB max upload on a 1 GB box; a large PDF through canvas/sharp could exhaust memory.	Lower the cap (e.g. 25–50 MB) or add per-request memory guards; monitor OOM.
S4	**LOW**	Reproducible-build gap: the lockfile is deleted at build/CI time to dodge a Windows-vs-Linux native-binary issue, so installs resolve fresh each time.	Generate a Linux-committed lockfile (or use npm overrides / a devcontainer) so production dependencies are pinned.
S5	**LOW**	SSH is open (to your IP) and there is no fail2ban / automatic security updates configured on the box.	Enable unattended-upgrades; consider fail2ban; keep SSH restricted to your IP.

5. Deployment Review (AWS EC2)

The migration is well executed for a low-cost, always-on target and fixes the Render sleeping / 502 problem. HTTPS is automatic via Caddy and was verified live. The stateless design means the box can be rebuilt freely. The concerns are about resilience and operations, not correctness.

Verified working

- HTTPS live with a valid certificate at the new domain (health endpoint returns `status: ok`).
- 3 GB swap active — prevents out-of-memory during the Next.js build on 1 GB RAM.
- Root volume expanded to 30 GB (was hitting a full 6.7 GB disk during build).
- Docker restart policy `unless-stopped` — containers survive a reboot.

Risks & recommendations

- **Single point of failure (MEDIUM):** one instance, no redundancy. If it dies the site is down until manual recovery. Acceptable for now; document a rebuild runbook (the new migration skill covers this).
- **No monitoring / alerting (MEDIUM):** nothing tells you if the box or a container goes down. Add a free uptime monitor (e.g. UptimeRobot) pinging `/api/health`, plus basic disk/RAM alerts.
- **No Elastic IP (LOW):** the public IP changes on stop/start, which breaks DNS. Allocate an Elastic IP (free while attached) so the address is stable.
- **Cost cliff (INFO):** EC2 free tier lasts 12 months, then ~$8–10/mo. Oracle Cloud Always-Free is a no-cost alternative if budget matters later.
- **Capacity (LOW):** 1 GB RAM limits concurrent heavy PDF operations; fine for light use, but a spike in compress/redact jobs could stall.

6. Code Quality, Features & Testing

- **Layering:** clear routes → services → lib separation; shared Zod schemas enforce one source of truth for request/response shapes.
- **Type safety:** full TypeScript with a passing typecheck gate in CI.

- **Tests:** 7 API suites (auth, files, pdf, edit, convert, redact, ocr-form) run via node:test in CI; conversion/OCR tests skip gracefully when native tools are absent. **Gap:** no end-to-end / browser tests of the web app.
- **CI:** GitHub Actions runs typecheck (hard gate), tests, and web build on every push to main; lint is advisory. Last run green.
- **Production runtime:** the API is executed with `tsx` (TypeScript directly) rather than a compiled build. It works, but a `tsc` build would start faster and remove tsx from the production dependency surface.
- **Schema management:** the container runs `prisma db push` on start rather than versioned migrations. The data-loss guard mitigates the main risk, but `migrate deploy` would give proper, reviewable schema history.

7. Performance

- Response compression (gzip) enabled for JSON and downloads.
- Health check sits above the rate limiter so monitoring pings don't consume the budget.
- 60-minute data retention job purges old files/activity — keeps storage and DB lean.
- Main constraint is the 1 GB instance: pdf rasterization (canvas) and image processing (sharp) are memory-hungry; heavy concurrent use will lean on swap and slow down. Vertical scaling (larger instance) is the simple lever if load grows.

8. Prioritized Action Plan

Priority	Action	Why
P0 — now	Rotate all exposed secrets (Neon, Supabase, Brevo, Google, DuckDNS) and revoke old GitHub PATs.	Plaintext exposure during setup; highest-impact, lowest-effort risk to close.
P0 — now	Add an uptime monitor on /api/health with email/SMS alerts.	You currently have no way to know the single box is down.
P1 — this week	Allocate an Elastic IP and attach it.	Stops the public IP from changing and silently breaking DNS on any stop/start.
P1 — this week	Run containers as a non-root user; enable unattended-upgrades on the box.	Reduces blast radius and keeps the OS patched automatically.
P2 — soon	Lower the upload size cap and/or add memory guards for heavy PDF ops.	Protects the 1 GB box from OOM under large or concurrent jobs.
P2 — soon	Commit a Linux-resolved lockfile; switch prod to a compiled build and versioned migrations.	Reproducible builds, faster start, reviewable schema history.
P3 — later	Add end-to-end tests for the core web flows (login, upload, each tool).	Catches regressions the API unit tests can't see.

9. Verdict

PDF Tool is production-ready for its current scale (89/100). It is live, secure in its fundamentals, feature-complete for a PDF utility, and now runs on affordable, always-on infrastructure with proper HTTPS. The remaining work is operational hardening rather than bug-fixing: rotate the exposed secrets,

add monitoring, stabilize the IP, and reduce single-box risk. Close the two P0 items today and the deployment moves from 'working' to 'operationally sound'.

Prepared by an automated senior-engineer audit pass on the live codebase and running deployment, 16 July 2026. Scores are indicative and relative to the previous re-audit.